

- FRX remediation-review change catalog
 - Purpose and limits
 - PCH-001 — Runtime session-signing key — worked partial
 - PCH-002 — Login query, password storage, and audit handling
 - PCH-003 — Report object authorization
 - PCH-004 — Analytics destination and object authorization
 - PCH-005 — Audit and error-output correction — extension

FRX remediation-review change catalog

Purpose and limits

These are bounded, course-authored excerpts from the proposed FRX `remediated` state. They let a reviewer inspect a correction claim and collect focused proof without receiving the complete instructor reference implementation. Each excerpt must be linked to a confirmed FN record before it enters the learner's remediation queue.

The excerpts do not establish deployment configuration, enterprise identity policy, secret-management policy, monitoring operation, full application coverage, final risk, residual-risk acceptance, or release approval. Use the **Lab 5 guide** (`labs/student-lab-guide.pdf`), **editable Day 2 record workbook** (`labs/day-2-record-templates.docx`), and `review-remediation.py` together.

PCH-001 — Runtime session-signing key — worked partial

- **Linked record:** FN1 / SV1 / `SAST-001` / `SECRET-001`.
- **Correction claim:** remove the fixed signing key from source, load a supplied runtime value when present, otherwise generate and retain a separate training-runtime value, and set the supplied cookie flags.
- **Modified surface:** `configure_app`; session configuration and the local runtime key file.

```

session_secret = os.environ.get("FRX_SESSION_SECRET")
if not session_secret:
    if not settings.session_key_path.exists():
        settings.session_key_path.write_text(
            secrets.token_urlsafe(48), encoding="utf-8"
        )
    session_secret = settings.session_key_path.read_text(encoding="utf-8").strip()

app.config.update(
    SECRET_KEY=session_secret,
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
)

```

Proof lanes: fixed assignment absent; runtime key created without disclosure; same runtime survives application restart; clean runtimes differ; ordinary login/session works; a request without a session is denied; cookie flags are present.

Required remaining-evidence statement: production secret source, access policy, file permissions/backup, rotation, revocation, previous-session transition, and deployed configuration remain unproved.

PCH-002 — Login query, password storage, and audit handling

- **Possible linked records:** a confirmed weak password-hash or login-query mechanism and related logging evidence.
- **Correction claim:** use a password-hash function intended for password storage, parameterize the username lookup, verify the stored hash, clear prior session state, and keep submitted passwords out of audit events.
- **Modified surface:** password hashing, login query, login decisions, returned identity, and login audit events.

```
def password_hash(password: str) -> str:
    return generate_password_hash(password, method="scrypt")

user = connection.execute(
    "SELECT id, username, password_hash, role, active FROM users WHERE username = ?",
    (username,),
).fetchone()
if user is None or not user["active"] or not check_password_hash(
    user["password_hash"], password
):
    _audit("login", "failed", username=username)
    return jsonify(error="Invalid username or password"), 401

session.clear()
session.update(user_id=user["id"], username=user["username"], role=user["role"])
```

Proof lanes: ordinary login succeeds; wrong credentials fail generically; punctuation in a missing username does not create a database error; stored hash scheme changes; audit records success/failure without the submitted password; ordinary session behavior remains.

Required remaining-evidence statement: production identity source, migration/forced-reset plan, password policy, rate limits, lockout, and monitoring ownership remain unproved.

PCH-003 — Report object authorization

- **Possible linked record:** CF1/FN authorization mechanism on `GET /reports/<report_id>`.
- **Correction claim:** enforce the stated partner-owner, analyst-assignment, and administrator rules after object lookup and before disclosure; audit both denial and success.
- **Modified surface:** shared `_can_read` decision and report-detail response path.

```
def _can_read(report) -> bool:
    role = session.get("role")
    user_id = session.get("user_id")
    return bool(
```

```

        role == "administrator"
    or (role == "partner" and report["owner_id"] == user_id)
    or (role == "analyst" and report["assigned_analyst_id"] == user_id)
)

if not _can_read(report):
    _audit("report-read", "denied", report_id=report_id, username=session["username"])
    return jsonify(error="Access denied"), 403

```

Proof lanes: anonymous denial; partner own/other records; analyst assigned/unassigned records; administrator access; missing object; denial and success audit events; no weakening of ordinary report reads.

Required remaining-evidence statement: policy-owner confirmation, identity/assignment-data integrity, and consistent enforcement on other interfaces remain unproved.

PCH-004 — Analytics destination and object authorization

- **Possible linked record:** a confirmed or narrowed attacker-influenced analytics-destination mechanism.
- **Correction claim:** accept a positive report identifier rather than a caller URL, authorize the referenced report, construct the destination from configured service origin plus application-selected path, validate response identity, and return bounded failures.
- **Modified surface:** `GET /analytics/fetch`, report lookup/authorization, outbound construction, response validation, and error handling.

```

report_id = request.args.get("report_id", type=int)
if report_id is None or report_id < 1:
    return jsonify(error="A positive report_id is required"), 400

if not _can_read(report):
    return jsonify(error="Access denied"), 403

destination = f"{_settings().analytics_base_url}/metrics/{report_id}"
with urllib.request.urlopen(destination, timeout=2) as remote:
    payload = remote.read(64 * 1024)

decoded = json.loads(payload)
if not isinstance(decoded, dict) or decoded.get("report_id") != report_id:
    return jsonify(error="Analytics response failed validation"), 502

```

Proof lanes: caller URL parameter rejected; positive owned identifier succeeds; other-owner identifier denied; invalid identifier and provider failures are bounded; only the configured origin/application path is called; response identifier must match.

Required remaining-evidence statement: production service identity, TLS/DNS/proxy/egress controls, provider authenticity/availability, and other outbound interfaces remain unproved.

PCH-005 — Audit and error-output correction — extension

- **Possible linked records:** confirmed secret-bearing audit or sensitive-error mechanisms.
- **Correction claim:** use structured event/outcome fields without submitted secrets and return a generic handled error while retaining an audit event.

- **Modified surface:** audit helper, login audit calls, demonstration error response, and global error behavior.

```
def _audit(event: str, outcome: str, **fields) -> None:
    safe_fields = " ".join(f"{key}={value}" for key, value in sorted(fields.items()))
    with _settings().audit_log_path.open("a", encoding="utf-8") as stream:
        stream.write(f"event={event} outcome={outcome} {safe_fields}".rstrip() + "\n")

_audit("demo-error", "handled", username=session["username"])
return jsonify(error="The requested training resource is unavailable."), 503
```

Proof lanes: authentication success/failure remains recorded; submitted password and configured key are absent; internal path is absent from the response; client failure is bounded; repository tests remain green.

Required remaining-evidence statement: central collection, access control, retention, tamper protection, alert routing, and deployed unhandled-exception behavior remain unproved.